

PROTOCOLO HTTP

Redes de Datos

2018

Universidad Nacional del Nordeste

Fa.CE.NA

Objetivos del Taller

- Afianzar y consolidar conocimientos del protocolo HTTP (Hypertext Transfer Protocol).
- Afianzar programación en socket.
- Afianzar conocimiento sobre servidores HTTP

Actividades del Taller

- Configuración de Servidor Apache 2.0 sobre Linux Debian 6.0
- Estudio del protocolo con Wireshark

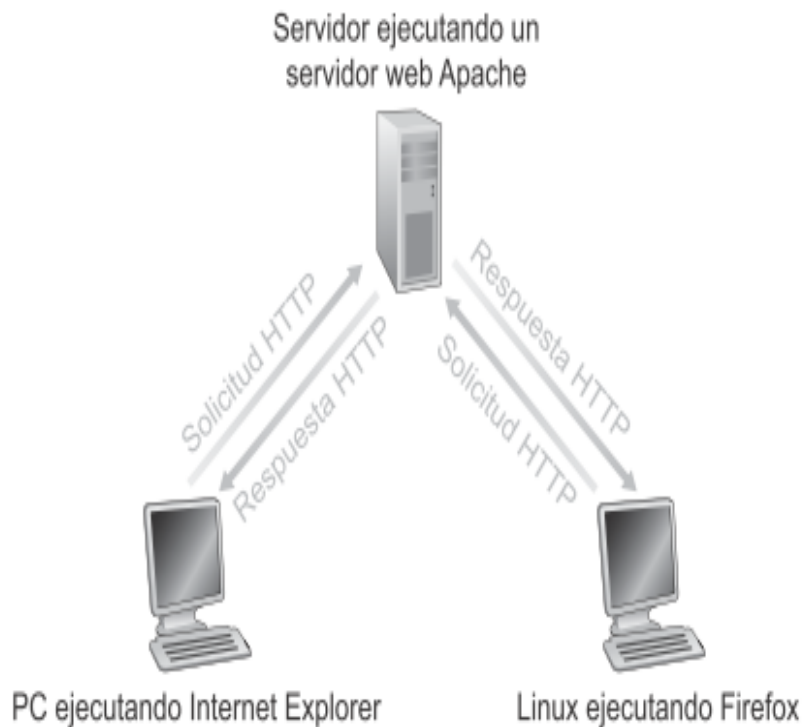
HTTP (Protocolo de Transferencia de Hipertexto)

- Usado para cada transferencia del Word Wide Web
- Desarrollado por [Word Wide Web Consortium](#)
- Fue desarrollado por [Tim Berners-Lee](#)
- La versión 1.1 especificada en el RFC2616
- La ultima versión es la 2.0 especificada en el RFC7540
- Es un protocolo de la Capa de Aplicación
- Al cliente que realiza la petición se lo conoce como User-Agent (UA)
- A la información transmitida se le llama recurso. Un recurso esta identificado en una URL
- Es un protocolo sin estado. Es por ello que se utiliza cookie o sesiones en las aplicaciones
- La Web fue la primera aplicación de Internet que atrajo la atención del público general.
- Cambió de manera dramática, y **continúa cambiando**, la forma en que las personas interactúan dentro y fuera de sus entornos de trabajo.

Tim Berners-Lee



HTTP (Protocolo de Transferencia de Hipertexto)



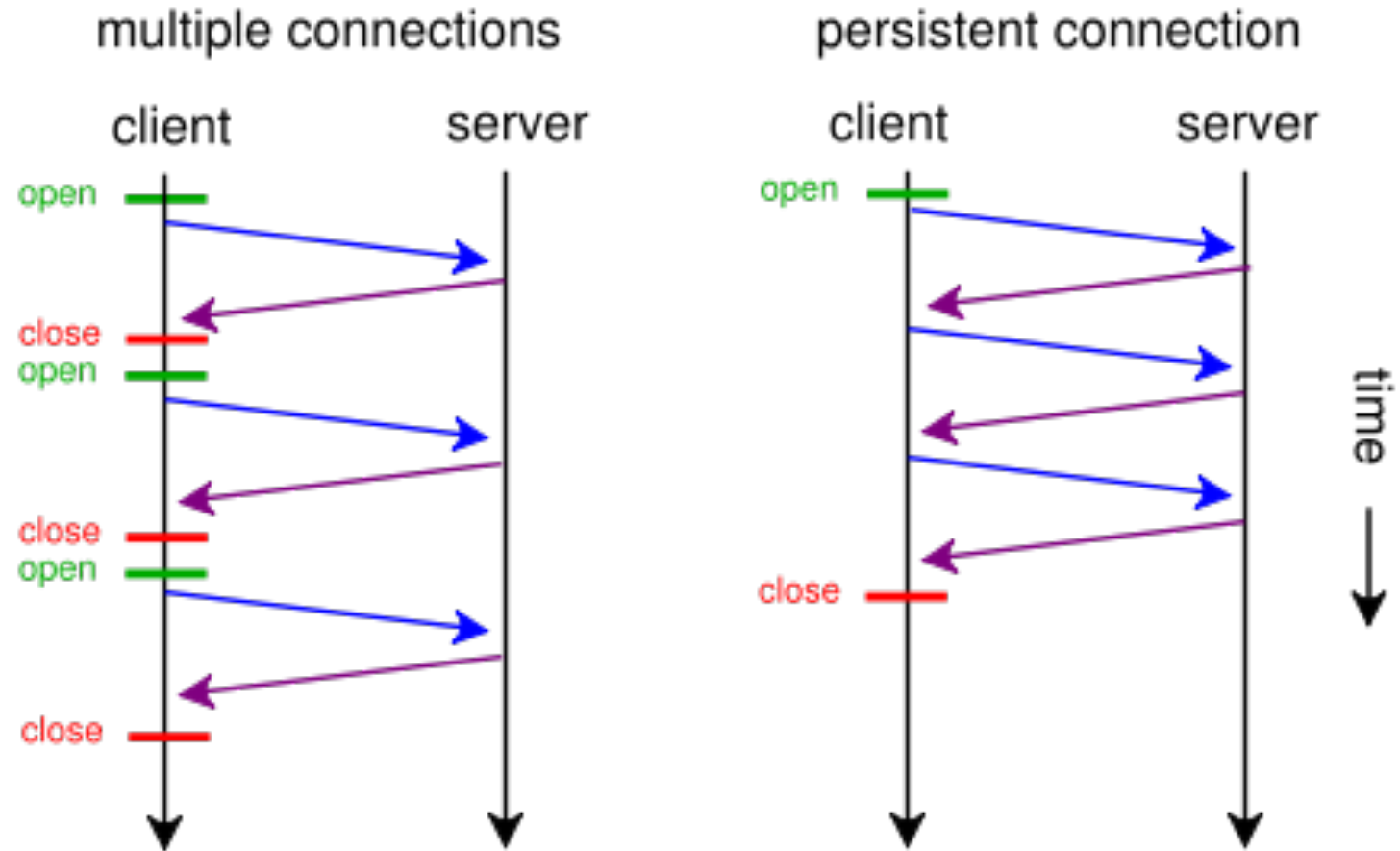
HTTP utiliza **TCP** como su protocolo de transporte subyacente El cliente HTTP primero inicia una conexión TCP con el servidor.

Una vez que la conexión se ha establecido, los procesos de navegador y de servidor acceden a TCP a través de sus interfaces de socket.

Conexiones Persistentes

- **Conexiones persistentes (Keep-Alive)**
- Antes de que esta característica existiese, se necesitaba una conexión TCP separada para cada petición
- HTTP en su última versión, proporciona la posibilidad de establecer sesiones de mayor duración de manera que se permiten múltiples peticiones sobre la misma conexión TCP.
- El modo por defecto de HTTP utiliza conexiones persistentes con procesamiento en cadena.
- Beneficio
 - Se abren menos conexiones TCP, lo que ahorra recursos (CPU, memoria, etc.).
 - Se reduce la latencia en peticiones al utilizar varias veces un canal ya abierto.

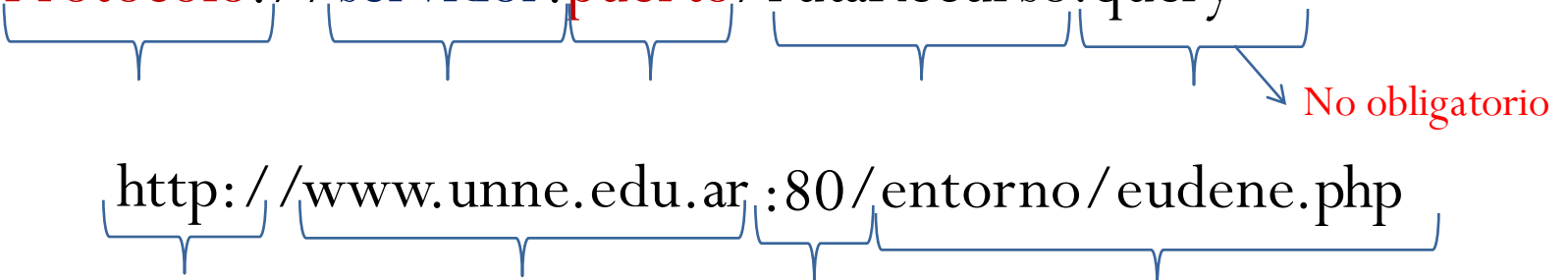
Conexiones persistentes



HTTP - Versiones

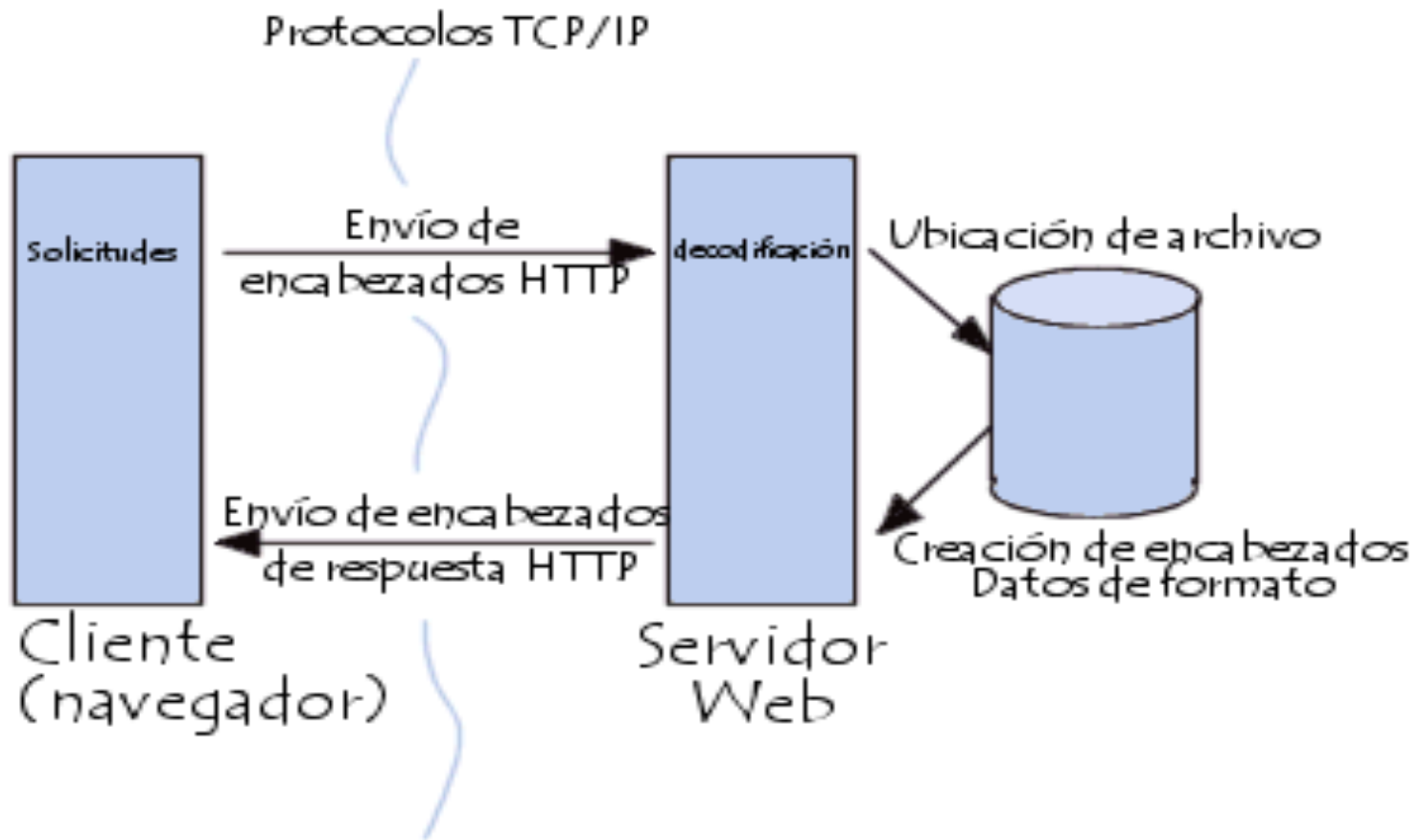
- HTTP/0.9
 - Obsoleta. Soporta solo el comando GET.
- HTTP/1.0 (mayo de 1996)
 - Esta es la primera revisión del protocolo que especifica su versión en las comunicaciones, y todavía se usa ampliamente.
- HTTP /1.1/(junio 1999)
 - Versión actual. Permite al cliente enviar múltiples peticiones a la vez por la misma conexión (pipelining) lo que hace posible eliminar el tiempo de Round-Trip delay por cada petición.
- HTTP 2.0

URL – Uniform resource locator

- Localizador de recurso uniforme
- Creado por [Tim Berners-Lee](#)
- Secuencia de carácter que permite, estandarizado que se usa para nombrar recursos en internet para su localización o identificación.
- No es sensible a mayuscula. www.unne.edu.ar = WWW.UNNE.EDU.AR
- **Protocolo:** // **servidor:** **puerto** / rutaRecurso?query


The diagram illustrates the components of a URL using the example `http://www.unne.edu.ar:80/entorno/eudene.php`. Brackets are placed under each part: `http`, `//`, `www.unne.edu.ar`, `:80`, `/entorno/eudene.php`, and `?`. A red arrow points from the `?` to the text "No obligatorio" (Not obligatory).

HTTP



Comunicación simple HTTP – Cliente Servidor

Imagen 1 - Comunicación simple de HTTP

Metodos

Comando	Descripción	Version
GET	Solicita el recurso ubicado en la URL especificada	v0.9;v1.0;v1.1
HEAD	Solicita el encabezado del recurso ubicado en la URL especificada. Similar al GET excepto que el server no retornar el cuerpo del mensaje en la respuesta	v1.0;v1.1
POST	Envía datos al programa ubicado en la URL especificada	v1.0;v1.1
PUT	Envía datos a la URL especificada	v1.1
DELETE	Borra el recurso ubicado en la URL especificada	v1.1
OPTIONS	Solicita los métodos que soporta el servidor	v1.1

Solicitud HTTP

Una línea de Solicitud: Es una línea que especifica el tipo de documento solicitado, el método que se aplicará y la versión del protocolo utilizada.

GET /index.html HTTP/1.1

Los campos del encabezado de solicitud: Líneas adicionales que dan información sobre la solicitud y/o el cliente (navegador, sistema operativo, etc.).

El cuerpo de la solicitud: es un conjunto de líneas opcionales que deben estar separadas de las líneas precedentes por una línea en blanco. Permiten que se envíen datos por un comando POST durante la transmisión de datos al servidor utilizando un formulario.

Sintaxis Solicitud : Formato General

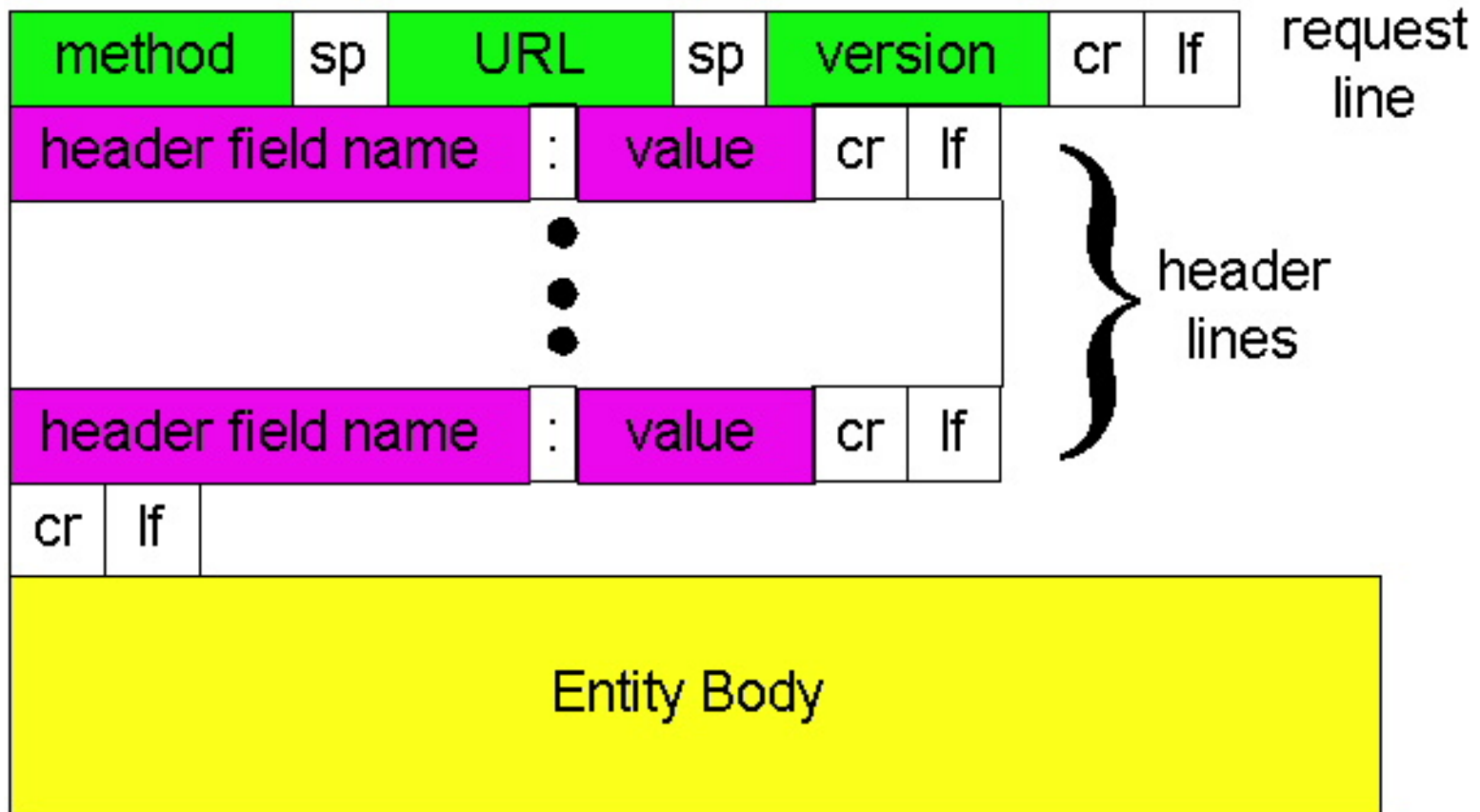


Imagen 3: Sintaxis Solicitud

Sintaxis Solicitud : Formato General

```
GET /unadireccion/pagina.html HTTP/1.1
```

```
Host: www.unaescuela.edu
```

```
Connection: close
```

```
User-agent: Mozilla/4.0
```

```
Accept-language: es
```

Host: www.unaescuela.edu especifica el host en el que reside el objeto.

Connection: close, el navegador está diciendo al servidor que no desea molestarse en trabajar con conexiones persistentes

User-agent: especifica el agente de usuario, es decir, el tipo de navegador que está haciendo la solicitud al servidor.

Accept-language: indica que el usuario prefiere recibir una versión en español del objeto

Encabezado Cliente

Nombre	Descripción
Accept	Tipo de contenido aceptado por el navegador (por ejemplo, <i>texto/html</i>). Consulte Tipos de MIME
Accept-Charset	Juego de caracteres que el navegador espera
Accept-Encoding	Codificación de datos que el navegador acepta
Accept-Language	Idioma que el navegador espera (de forma predeterminada, inglés)
Authorization	Identificación del navegador en el servidor
Content-Encoding	Tipo de codificación para el cuerpo de la solicitud
Content-Language	Tipo de idioma en el cuerpo de la solicitud
Content-Length	Tamaño del cuerpo de la solicitud en bytes
Content-Type	Tipo de contenido del cuerpo de la solicitud (por ejemplo, <i>texto/html</i>). Consulte Tipos de MIME
Date	Fecha en que comienza la transferencia de datos
Forwarded	Utilizado por equipos intermediarios entre el navegador y el servidor

Encabezado Cliente

Nombre	Descripción
Link	Vínculo entre dos direcciones URL
Orig-URL	Dirección URL donde se originó la solicitud
Referer	Dirección URL desde la cual se realizó la solicitud
User-Agent	Cadena con información sobre el cliente, por ejemplo, el nombre y la versión del navegador y el sistema operativo

Respuesta HTTP

- **Una línea de estado:** Es una línea que especifica la versión del protocolo utilizada, el código de estado y la explicación.

HTTP/1.1 200 OK

- **Los campos del encabezado de respuesta:** Líneas opcionales que permiten aportar información adicional sobre la respuesta y/o el servidor. Cada una de estas líneas está compuesta por un nombre que califica el tipo de encabezado, seguido por dos puntos (:) y por el valor del encabezado
- **El cuerpo de la respuesta:** Contiene el documento solicitado.

Sintaxis Respuesta: Formato General

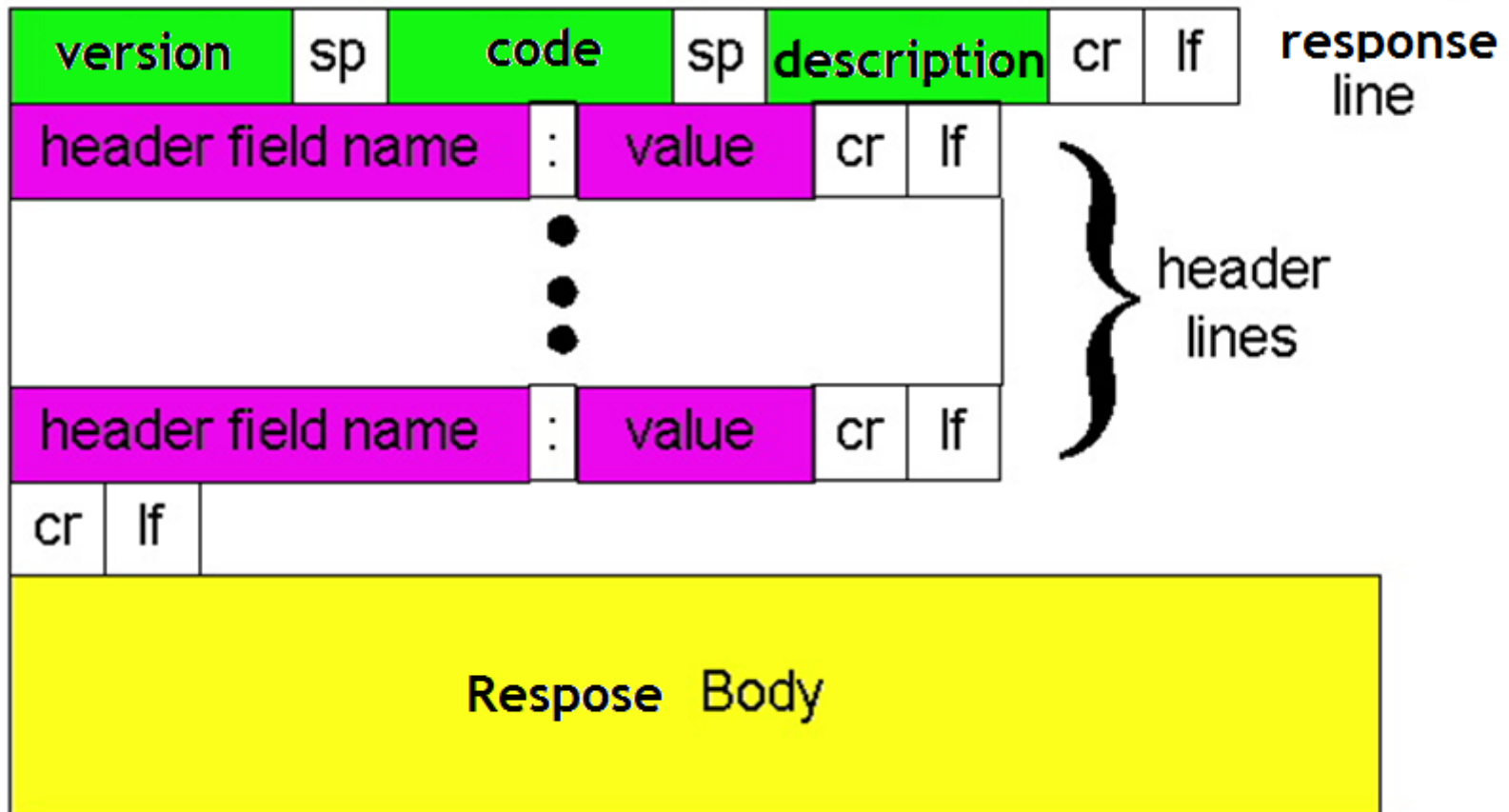


Imagen 4: Sintaxis Respuesta

Sintaxis Solicitud : Formato General

```
HTTP/1.1 200 OK
Connection: close
Date: Sat, 07 Jul 2007 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Sun, 6 May 2007 09:23:24 GMT
Content-Length: 6821
Content-Type: text/html
(datos datos datos datos datos ...)
```

Connection: close para indicar al cliente que va a cerrar la conexión TCP después de enviar el mensaje. La línea de cabecera

Date: indica la hora y la fecha en la que se creó la respuesta HTTP y fue enviada por el servidor.

Server: indica que el mensaje fue generado por un servidor web Apache;

Last-Modified: resulta fundamental para el almacenamiento en caché del objeto

Content-Length: especifica el número de bytes del objeto que está siendo enviado.

Content-Type: indica que el objeto especificado en el cuerpo de entidad es texto HTML.

Encabezados Servidor

Content-Encoding	Tipo de codificación para el cuerpo de la respuesta
Content-Language	Tipo de idioma en el cuerpo de la respuesta
Content-Length	Extensión del cuerpo de la respuesta
Content-Type	Tipo de contenido del cuerpo de la respuesta (por ejemplo, <i>texto/html</i>). Consulte Tipos de MIME
Date	Fecha en que comienza la transferencia de datos
Expires	Fecha límite de uso de los datos
Forwarded	Utilizado por equipos intermediarios entre el navegador y el servidor
Location	Redireccionamiento a una nueva dirección URL asociada con el documento
Server	Características del servidor que envió la respuesta

Código de respuesta

Código	Mensaje	Descripción
10x	Mensaje de información	Estos códigos no se utilizan en la versión 1.0 del protocolo
20x	Éxito	Estos códigos indican la correcta ejecución de la transacción
200	OK	La solicitud se llevó a cabo de manera correcta
201	CREATED	Sigue a un comando POST e indica el éxito, la parte restante del cuerpo indica la dirección URL donde se ubicará el documento creado recientemente.
202	ACCEPTED	La solicitud ha sido aceptada, pero el procedimiento que sigue no se ha llevado a cabo
203	PARTIAL INFORMATION	Cuando se recibe este código en respuesta a un comando de GET indica que la respuesta no está completa.
204	NO RESPONSE	El servidor ha recibido la solicitud, pero no hay información de respuesta
205	RESET CONTENT	El servidor le indica al navegador que borre el contenido en los campos de un formulario

Código de respuesta

Código	Mensaje	Descripción
30x	Redirección	Estos códigos indican que el recurso ya no se encuentra en la ubicación especificada
301	MOVED	Los datos solicitados han sido transferidos a una nueva dirección
302	FOUND	Los datos solicitados se encuentran en una nueva dirección URL, pero, no obstante, pueden haber sido trasladados
303	METHOD	Significa que el cliente debe intentarlo con una nueva dirección; es preferible que intente con otro método en vez de GET
304	NOT MODIFIED	Si el cliente llevó a cabo un comando GET condicional (con la solicitud relativa a si el documento ha sido modificado desde la última vez) y el documento no ha sido modificado, este código se envía como respuesta.

Código de respuesta

Código	Mensaje	Descripción
40x	Error debido al cliente	Estos códigos indican que la solicitud es incorrecta
400	BAD REQUEST	La sintaxis de la solicitud se encuentra formulada de manera errónea o es imposible de responder
401	UNAUTHORIZED	Los parámetros del mensaje aportan las especificaciones de formularios de autorización que se admiten. El cliente debe reformular la solicitud con los datos de autorización correctos
402	PAYMENT REQUIRED	El cliente debe reformular la solicitud con los datos de pago correctos
403	FORBIDDEN	El acceso al recurso simplemente se deniega
404	NOT FOUND	Un clásico. El servidor no halló nada en la dirección especificada. Se ha abandonado sin dejar una dirección para redireccionar... :)

Código de respuesta

Código	Mensaje	Descripción
50x	Error debido al servidor	Estos códigos indican que existe un error interno en el servidor
500	INTERNAL ERROR	El servidor encontró una condición inesperada que le impide seguir con la solicitud (una de esas cosas que les suceden a los servidores...)
501	NOT IMPLEMENTED	El servidor no admite el servicio solicitado (no puede saberlo todo...)
502	BAD GATEWAY	El servidor que actúa como una puerta de enlace o proxy ha recibido una respuesta no válida del servidor al que intenta acceder
503	SERVICE UNAVAILABLE	El servidor no puede responder en ese momento debido a que se encuentra congestionado (todas las líneas de comunicación se encuentran congestionadas, inténtelo de nuevo más adelante)
504	GATEWAY TIMEOUT	La respuesta del servidor ha llevado demasiado tiempo en relación al tiempo de espera que la puerta de enlace podía admitir (excedió el tiempo asignado...)

Dialogo HTTP

Cliente

GET /index.html HTTP/1.1 <crLf>

Host: www.unne.edu.ar <crLf>

User-Agent: nombre-cliente <crLf>

[Línea en blanco] <crLf>

Servidor

HTTP/1.1 200 OK <crLf>

Date: Mon, 20 May 2013 06:02:52 GMT <crLf>

Server: Apache/2.0.51 (Fedora) <crLf>

Content-Length: 1221 <crLf>

[Línea en blanco] <crLf>

<html>

<title> Universidad Nacional del Nordeste</title>

<body> ... </body> </html>

CR: Retorno de carro.

LF: Avance de Linea

Construyendo un simple Servidor Web con c# - Visual Studio 2012

```
Class MyTcpListner {  
    TcpListener server = null;  
    try  
    {  
        Int32 port = 4545;  
        IPAddress localAddr = IPAddress.Parse("127.0.0.1");  
        server = new TcpListener(localAddr, port);  
        server.Start();  
    }  
    while (true) {  
        Console.WriteLine("Esperando conexión en el Servidor Web ");  
        TcpClient client = server.AcceptTcpClient(); //Aceptamos la conexión entrante  
        ManejadorPetición manejarPetición = new ManejadorPetición(client);  
    }  
}
```

Construyendo un simple Servidor Web con c# - Visual Studio 2012

Class ManejadorPeticion

```
public ManejadorPeticion(TcpClient cliente){  
    clienteWeb = cliente;  
    Thread ctThread = new Thread(doWeb);  
    ctThread.Start();  
}
```

Hilo



```
private void doWeb(){  
    while ((true))  
    {  
        try  
        {  
            NetworkStream networkStream = clienteWeb.GetStream(); //Obtenemos el Stream  
            networkStream.Read(bytesFrom, 0, bytesFrom.Length); //Leemos el Stream y guardamos  
            petitionBrowser = System.Text.Encoding.ASCII.GetString(bytesFrom);  
            Peticion petition = new Peticion(petitionBrowser);  
        }  
    }  
}
```

Construyendo un simple Servidor Web con c# - Visual Studio 2012

```
class Peticion {  
[Variables]  
public Peticion(string petitionBrowser)  
{  
    string[] header = petitionBrowser.Split('\n');  
    method = header[0].Split(' ')[0];  
    file = header[0].Split(' ')[0];  
    fileExist(file);  
}
```

Construyendo un simple Servidor Web con c# - Visual Studio 2012

```
public void fileExist(string file) {  
    if(file.Equals("/")) {  
        file= "/index.html";  
    }  
    if (File.Exists("web" + file) == false) {  
        sCodigo = "404 Not Found";  
        FileContent = "<html><head><title>404 Not  
Found</title></head><body><h1>Not Found</h1><p>The requested URL  
/asdsada was not found on this server.</p><p>Additionally, a 404 Not Found error was  
encountered while trying to use an ErrorDocument to handle the  
request.</p></body></html>";  
        iTotBytes = FileContent.Length;  
    } else  
    { sCodigo = "200 OK"; // Seteo Codigo en 200
```

Construyendo un simple Servidor Web Python

```
request = self.recv().strip()

#Verificamos el nro de palabras en la solicitud
if self.contar_palabras(request) == 3 or self.contar_palabras(request) == 1:
    #Obtenemos la primer palabra (o la unica) de la solicitud
    method = self.obtener_palabra(request, 0)

    #Verificamos que sea un atributo de la clase y que si es llamable
    if hasattr(self, method) and callable(getattr(self, method)):

        #Si es una una pablabra y no es ningún metodo HTTP
        if self.contar_palabras(request) == 1 and (not self.es_method_http(method)):
            #Llamamos al metodo
            getattr(self, method)()
            self.add_log(method, "200 OK")

        #Si es un metodo HTTP y el nro de palabras de solicitud es 3
        elif self.es_method_http(method) and self.contar_palabras(request) == 3:
            version_HTTP = self.obtener_palabra(request, 2)

            #Si la version del protocolo enviado es valido
            if version_HTTP in self.version_http_supported:
                #Llamamos al metodo y pasamos la solicitud
                getattr(self, method)(request)
                self.add_log(method, "200 OK")
            else:
                self.send_error_http(request, 400, self.encabezado(400,"text/html"))
```

Construyendo un simple Servidor Web Python

```
def GET(self, p_request):
    resource = self.obtener_palabra(p_request, 1)
    version_server = self.obtener_palabra(p_request, 2)
    if resource == "/":
        resource = "/index.html"

    try:

        if resource.endswith(".html"):
            mimetype = 'text/html'
        if resource.endswith(".css"):
            mimetype = 'text/css'

        file = open(curdir + sep + resource)
        body = file.read()
        file.close()

        head = self.encabezado(200, mimetype)
        msj = str(head) + str(body) + "\n\n"
        self.send(msj)

    except IOError:
        head_err = self.encabezado(404, "text/html")
        self.send_error_http(p_request, 404, head_err)

#end if

#end GET
```

Bibliografia

- RFC2616 - Hypertext Transfer Protocol -- HTTP/1.1. [Fecha de consulta: 22 mayo 2013] . Disponible en <http://www.ietf.org/rfc/rfc2616.txt>
- Localizador Uniforme de recursos - [Fecha de consulta: 25 mayo 2013]
[http://es.wikipedia.org/wiki/Localizador uniforme de recursos](http://es.wikipedia.org/wiki/Localizador_uniforme_de_recursos)
- Uniform resource identifier - - [Fecha de consulta: 25 mayo 2013]
[http://en.wikipedia.org/wiki/Uniform resource identifier](http://en.wikipedia.org/wiki/Uniform_resource_identifier)
- Protocolo HTTP - [Fecha de consulta: 1 Junio 2013]
<http://es.kioskea.net/contents/264-el-protocolo-http>
- Protocolo HTTP - [Fecha de consulta: 1 Junio 2013]
[http://es.wikipedia.org/wiki/Hypertext Transfer Protocol#URL](http://es.wikipedia.org/wiki/Hypertext_Transfer_Protocol#URL)
- Google Android User Agent String - Fecha de consulta: 11 Junio 2013]
<http://www.gtrifonov.com/2011/04/15/google-android-user-agent-strings-2/>